المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵎⵙⵙⵓⵔⵉⵏ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⵙⵙⵓⵔⵉⵏ ⵜⵉⵏⴰⵏⵜⵉⵏ
Ecole Nationale des Sciences Appliquées de Tétouan

École Nationale des Sciences Appliquées de Tétouan
Département d'Intelligence Artificielle et de Digitalisation IAD
Science des Données, Big Data et IA

MISE EN PLACE D'UN MINI DATA WAREHOUSE AVEC PENTAHO PDI

Extraction, transformation et chargement des données dans PostgreSQL

Préparé par:
Yassir Adila
Class: BDIA2

Supervisé par:
Pr. Nisrine EL AYAT

Année académique: 2025–2026

Contents

1 Méthodologie suivie **2**
1.1 Extraction et Nettoyage (Dimensions) 2
1.2 Chargement de la Table de Faits 2

2 Choix techniques **2**
2.1 Base de données (PostgreSQL) 2
2.2 Stratégies ETL dans Pentaho 3
2.2.1 Transformation Étudiants (Nettoyage complexe en cascade) . . . 3
2.2.2 Transformation Cours (Validation simple) 3
2.2.3 Transformation Faits Examens (Intégrité Référentielle) 4

3 Problèmes rencontrés et solutions **4**

4 Preuves du résultat final **4**
4.1 Flux ETL (Pentaho Data Integration) 4
4.2 Données chargées en Base (PostgreSQL / pgAdmin) 5

1 Méthodologie suivie

L'objectif de ce TP était de concevoir une chaîne ETL complète pour alimenter un entrepôt de données (Data Warehouse) universitaire à partir de fichiers plats CSV. J'ai suivi une architecture en étoile composée de deux dimensions (dim_etudiant, dim_cours) et une table de faits (fact_examens).

1.1 Extraction et Nettoyage (Dimensions)

Dimension Étudiants : J'ai extrait les données brutes de etudiants.csv. Une chaîne de validation rigoureuse a été mise en place pour filtrer les enregistrements invalides (âge <17, format de sexe incorrect, valeurs nulles, doublons...).

- **Résultat :** 90 étudiants valides ont été insérés dans la base PostgreSQL. Les rejets ont été redirigés vers un fichier Excel d'audit (*etudiants_dirty.xlsx*).

Dimension Cours : Le fichier cours.csv a été normalisé et chargé directement.

- **Résultat :** 8 cours disponibles (Types : Obligatoire/Optionnel).

1.2 Chargement de la Table de Faits

Faits Examens : Pour garantir l'intégrité référentielle, j'ai extrait examens.csv et j'ai effectué deux jointures avec les données valides/ netoyées des étudiants et des cours. Seuls les examens liés à des étudiants/cours existants dans notre dimension nettoyée ont été conservés.

- **Résultat :** 720 examens chargés avec succès dans fact_examens.

2 Choix techniques

2.1 Base de données (PostgreSQL)

J'ai choisi PostgreSQL pour sa robustesse et sa gestion stricte des contraintes. Le script SQL de création intègre des règles de qualité directement au niveau de la base :

Contraintes de domaine (CHECK) : j'ai forcé l'intégrité des données (ex: `CHECK (age > 17)` et `CHECK (sexe IN ('M','F'))`) pour agir comme une seconde barrière de sécurité après l'ETL.

Clés étrangères (FOREIGN KEY) : Définies dans `fact_examens` pour empêcher l'insertion de lignes orphelines.

2.2 Stratégies ETL dans Pentaho

J'ai découpé le processus en trois transformations distinctes pour assurer la modularité et la maintenabilité du projet.

2.2.1 Transformation Étudiants (Nettoyage complexe en cascade)

Cette transformation est la plus critique car le fichier source contient de nombreuses anomalies. j'ai opté pour une approche de filtrage séquentiel :

Dédoublonnage et préservation des données : J'ai volontairement évité l'étape *Unique Rows*, car elle supprime les doublons de manière arbitraire, ce qui risque d'entraîner une perte d'information (impossibilité de savoir quelle version de la ligne conserver). Pour ne pas perdre de données, j'ai mis en place une logique d'audit :

- Tri des données (*Sort Rows*) suivi d'un regroupement par ID (*Group By*) pour compter le nombre d'occurrences d'une ligne.
- Filtrage (*Filter Rows*) pour identifier les ID dont le compte est strictement supérieur à 1 (*count>1*).
- Redirection de ces doublons vers notre fichier d'erreurs Excel.

Cette méthode permet de conserver les données problématiques afin qu'elles soient corrigées manuellement lors d'un audit, plutôt que d'être supprimées silencieusement.

Caractères spéciaux : De plus, pour certains champs contenant des noms ou des textes, nous avons détecté des caractères spéciaux indésirables. Pour les supprimer, nous avons utilisé l'étape Remplacer dans Pentaho avec des expressions régulières (RegEx) afin de nettoyer les colonnes de type String avant leur chargement dans la table dim_etudiant.

Filtrage en cascade : j'ai enchaîné les étapes *Filter Rows* :

- *Null Values Filter* : Vérifie la présence des données obligatoires.
- *Age Filter* : Valide que l'âge est > 17 ans.
- *Gender Filter* : Vérifie que le sexe correspond à 'M' ou 'F'.

Gestion des types : L'étape *Data Types Validation (Select Values)* standardise les métadonnées (conversion String vers Entier pour l'âge) avant le chargement dans la table `dim_etudiant`, ainsi qu'elle m'a aidé dans le choix des colonnes qui vont passer dans le reste du workflow.

2.2.2 Transformation Cours (Validation simple)

Le fichier source étant relativement propre, j'ai appliqué une transformation légère : extraction, vérification du format des colonnes (typage), et chargement direct dans `dim_cours`.

2.2.3 Transformation Faits Examens (Intégrité Référentielle)

Le défi ici est de ne charger que les examens valides.

Jointure de vérification : J'ai utilisé un *Merge Join (INNER)* entre le flux des examens et le flux des étudiants valides, ensuite une autre jointure entre la première sortie et la dimension dim_cours.

Logique de rejet : Cette jointure agit comme un filtre *Look-up*. Si l'ID étudiant et l'ID du cours ne trouvent pas de correspondance, la ligne est dirigée vers le flux d'erreur (*Invalid Exam Filter*) et exportée vers Excel. Seules les lignes valides sont chargées dans fact_examens.

3 Problèmes rencontrés et solutions

Défi principal : La centralisation et la traçabilité des rejets multiples

Le problème : Notre transformation génère plusieurs flux de rejet distincts (Doublons, Valeurs nulles, Age invalide, Sexe incorrect). Initialement, nous ne pouvions pas placer une étape Excel Writer à la sortie de chaque filtre pour cibler le même fichier etudiants_dirty.xlsx. Cela est techniquement impossible : tenter d'écrire simultanément dans le même fichier depuis plusieurs étapes provoque des conflits d'accès concurrents (le fichier est verrouillé par le premier processus d'écriture, faisant échouer les autres). De plus, générer un fichier Excel différent pour chaque type d'erreur aurait rendu l'audit dispersé et fastidieux.

La solution : j'ai ajouté une étape *Add Constants* qui crée un champ `Type_Erreur` (ex: "Age invalide", "Sexe incorrect"). Tous les flux d'erreurs convergent ensuite vers une étape *Dummy* avant d'être écrits dans etudiants_dirty.xlsx à l'aide d'un seul unique *Excel writer step* au lieu de plus de 5 Excel writers avant.

4 Preuves du résultat final

4.1 Flux ETL (Pentaho Data Integration)

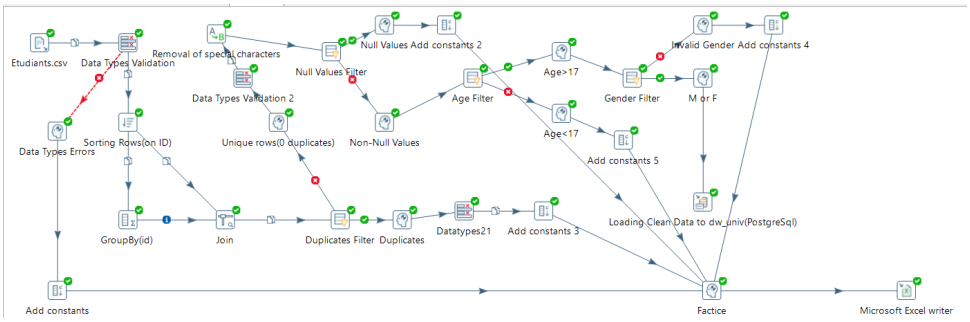


Figure 1: Transformation et nettoyage de la dimension Étudiant

On observe la logique de filtrage en cascade et la centralisation des rejets.

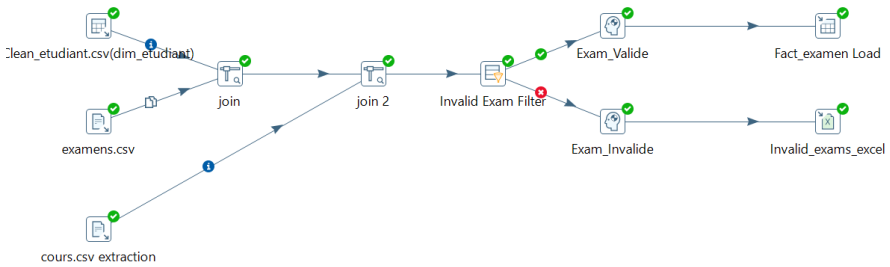


Figure 2: Chargement de la table de faits Examens

La jointure assure que seules les données cohérentes sont chargées.

4.2 Données chargées en Base (PostgreSQL / pgAdmin)

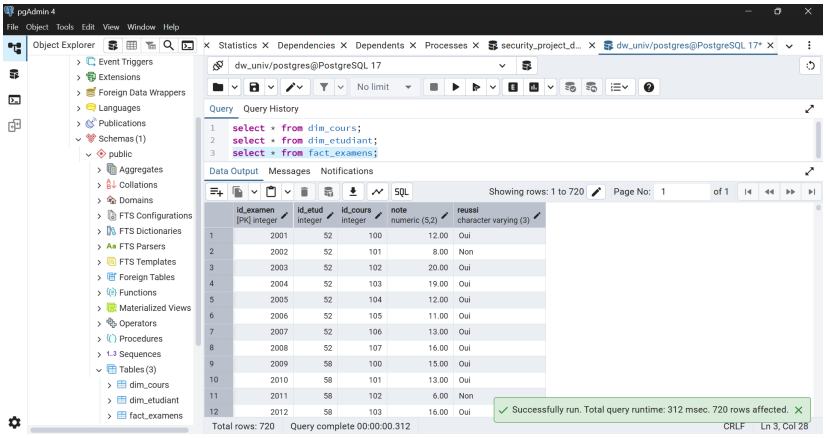


Figure 3: Table fact_examens

La requête confirme le chargement de 720 enregistrements validés.

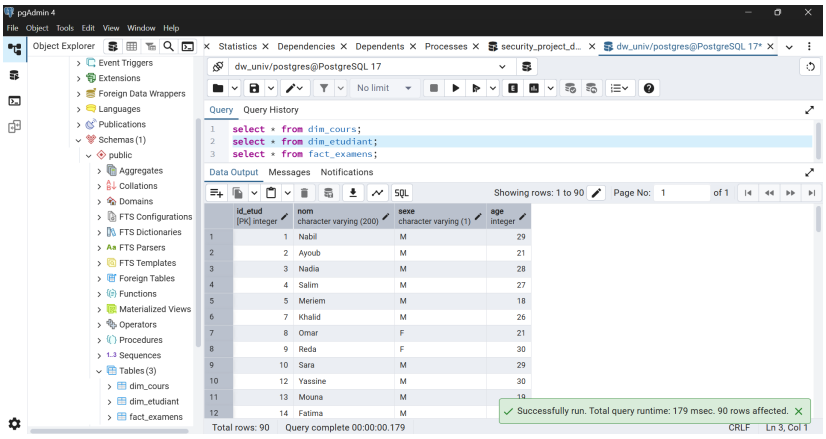
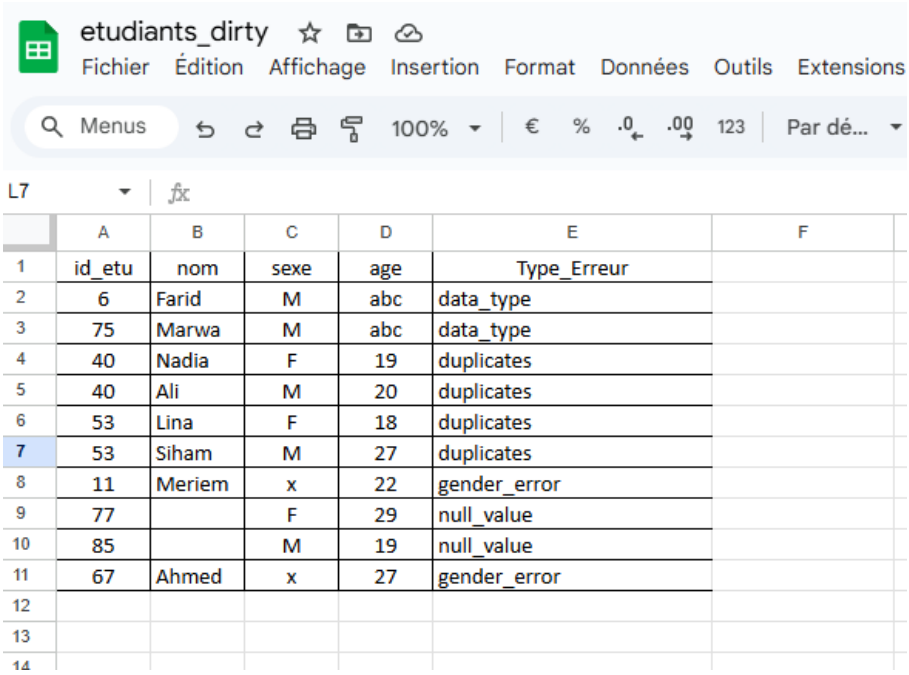


Figure 4: Contenu de dim_etudiant

Seuls les 90 étudiants respectant les critères ont été intégrés.



	A	B	C	D	E	F
1	id_etu	nom	sexe	age	Type_Erreur	
2	6	Farid	M	abc	data_type	
3	75	Marwa	M	abc	data_type	
4	40	Nadia	F	19	duplicates	
5	40	Ali	M	20	duplicates	
6	53	Lina	F	18	duplicates	
7	53	Siham	M	27	duplicates	
8	11	Meriem	x	22	gender_error	
9	77		F	29	null_value	
10	85		M	19	null_value	
11	67	Ahmed	x	27	gender_error	
12						
13						

Figure 5: Le fichier d'erreurs etudiants_dirty.xlsx